

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.TTNT

SINH VIÊN THỰC HIỆN

Mã sinh viên: 25522051

Họ và tên: Nguyễn Khánh Vân

TÊN ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG GAME MEOWSWEEPER

CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

Dự án là một ứng dụng trò chơi giải đố logic được xây dựng dựa trên tựa game "Dò Mìn" (Minesweeper) kinh điển. Mục tiêu của trò chơi là sử dụng tư duy suy luận để tìm ra vị trí toàn bộ mìn, cấm cờ đánh dấu và lật mở các vùng an toàn. Ứng dụng không chỉ tái hiện lại lối chơi truyền thống mà còn được nâng cấp với nhiều tính năng mới nhằm giảm thiểu yếu tố may rủi và mang tính giáo dục cao. Điểm nhấn đặc biệt của phiên bản này là việc tích hợp Hệ thống mạng sống (Lives system) cho phép người chơi mắc sai lầm, cùng Hệ thống gợi ý thông minh (Smart Hints) giúp rèn luyện tư duy logic.

b. Cốt truyện: Chiến dịch bảo vệ kho báu của Meo Meo

Meo Meo là một bé mèo trắng nhỏ nhắn và vô cùng háu ăn. Gia tài lớn nhất của em là những chú cá thơm ngon được cất giấu cẩn thận khắp nơi. Thế nhưng, thế giới loài mèo đâu có đơn giản! Xung quanh luôn có những gã mèo hoang ranh mãnh rình rập, chỉ chờ Meo Meo sơ hở là lao ra cướp sạch bữa ăn ngon.

Đến với Meowsweeper, người chơi sẽ hóa thân thành "vệ sĩ" đắc lực của Meo Meo. Nhiệm vụ của bạn là dựa vào các con số gợi ý trên màn hình để suy luận và đánh dấu chính xác vị trí những chú cá đang được giấu. Hãy cẩn thận trong từng bước đi! Nếu bạn lỡ tay mở trực tiếp vào ô giấu cá mà không đánh dấu bảo vệ, bọn mèo hoang sẽ nâng tay trên, chỉ để lại một bộ xương khô,

khiến Meo Meo buồn bã mà rơi mất một nhịp tim. Cùng trở tài suy luận logic để bảo vệ kho tàng cá và đừng để bụng bé Meo Meo réo lên vì đói nhé!

c. Link github

<https://github.com/khanhvanng/minesweeper>

d. Các CTDL và giải thuật đã được sử dụng:

- Lớp Cell (Đối tượng Ô vuông)
 - + What: Là một kiểu dữ liệu tự định nghĩa đại diện cho một ô duy nhất trên bàn cờ. Đối tượng này lưu trữ các trạng thái độc lập bao gồm: isMine (có mìn không), isRevealed (đã mở chưa), isFlagged (đã cấm cờ chưa) và neighbors (số mìn hiện diện ở 8 ô xung quanh).
 - + Why: Việc áp dụng tính đóng gói (encapsulation) trong lập trình hướng đối tượng giúp tập hợp toàn bộ thông tin của một ô vào một đối tượng duy nhất, làm cho mã nguồn gọn gàng và dễ quản lý. Phương pháp này tối ưu hơn việc sử dụng nhiều mảng hai chiều song song để lưu trữ từng trạng thái rời rạc.
- Lớp Board:
 - + What: Bàn cờ được tổ chức dưới dạng một ma trận hai chiều có kích thước hàng nhân cột (rows x cols) tương ứng với cấp độ khó của trò chơi.
 - + Why: Do bản chất trò chơi là một mạng lưới dạng lưới (grid-based), mảng 2 chiều là cấu trúc dữ liệu mô phỏng không gian vật lý này một cách trực quan và chính xác nhất. Hơn nữa, nó cho phép truy xuất và cập nhật dữ liệu cực kỳ nhanh chóng với độ phức tạp thời gian là $O(1)$ thông qua tọa độ.
- Cài đặt rải mìn sau khi mở ô đầu tiên:
 - + What: Hệ thống sẽ không sinh mìn ngay lúc khởi tạo màn chơi mà chờ người chơi click chuột vào ô đầu tiên. Điều này đảm bảo mìn sẽ không được rải vào ô vừa click và 8 ô lân cận xung quanh nó.
 - + Why: Giải quyết yêu cầu "click đầu tiên luôn an toàn". Việc tạo ra vùng cách ly 3x3 giúp ô trung tâm luôn có số mìn xung quanh là 0, từ đó kích hoạt hiệu ứng mở loang, cung cấp đủ dữ kiện suy luận ban đầu và loại bỏ vấn đề người chơi thua ngay từ lần nhấn đầu tiên.
- Thuật toán mở ô (Flood Fill/DFS):

- + What: Khi người chơi lật trúng một ô trống (không có mìn và neighbors = 0), thuật toán Tìm kiếm theo chiều sâu (DFS - Đệ quy) sẽ được kích hoạt để tự động mở lan truyền các ô an toàn lân cận. Quá trình loang sẽ dừng lại khi chạm đến biên của bàn cờ hoặc chạm đến các ô có chứa con số chỉ báo.
- + Why: Flood Fill là bài toán kinh điển và tối ưu nhất để xử lý việc duyệt qua các vùng không gian liên thông trên lưới 2D. Thuật toán này tạo ra cơ chế "mở dây chuyền", giúp tiết kiệm thao tác cho người chơi và mang lại trải nghiệm giải đố thỏa mãn.
- Hệ thống mạng sống:
 - + What: Khác với luật "chạm mìn là chết" cổ điển, trò chơi cung cấp một số lượng mạng sống nhất định. Nếu mở trúng mìn, số mạng sẽ bị trừ đi, quả mìn đó được bộc lộ và trò chơi chỉ kết thúc khi mạng sống lùi về 0.
 - + Why: Cơ chế này cho phép người chơi được phép mắc sai lầm, hỗ trợ cho những người mới tiếp cận game giải đố dễ dàng hơn mà không phải chơi lại từ đầu liên tục.
- Hệ thống gợi ý:
 - + What: Máy tính được dạy các quy tắc suy luận logic cơ bản (như xét giao tập hợp, quy tắc đủ cò/đủ mìn) để tự tìm ra ô an toàn hoặc ô chứa mìn. Khi sử dụng, hệ thống sẽ highlight khu vực liên quan kèm theo thông báo giải thích cách thức suy luận.
 - + Why: Biến trò chơi từ một hình thức giải trí đơn thuần thành một công cụ rèn luyện não bộ, giúp đỡ người chơi vượt qua bế tắc và thấu hiểu cách tư duy logic thay vì phải đoán mò.

2. Quá trình thực hiện

a. Tuần 1:

- Khai báo cấu trúc dữ liệu Cell, Board.
- Cài đặt rải mìn theo điều kiện (không rải vào ô đầu tiên đã chọn và 8 ô lân cận).
- Tính toán số mìn lân cận của từng ô trên bảng chơi.
- Cài đặt thuật toán mở ô.
- Khởi tạo repository và cập nhật lên Github.

b. Tuần 2:

- Cài đặt thư viện Kivy, tạo giao diện game cơ bản để tương tác.
- Tạo hệ thống mạng sống.
- Cài đặt điều kiện thắng thua.

c. Tuần 3:

- Cài đặt hệ thống gợi ý cho người chơi bằng AI suy luận.
- Cài đặt UI để tích hợp gợi ý vào trò chơi.

d. Tuần 4:

- Hoàn thiện giao diện game.
- Cải tiến thuật toán để tăng tốc độ và sửa lỗi.

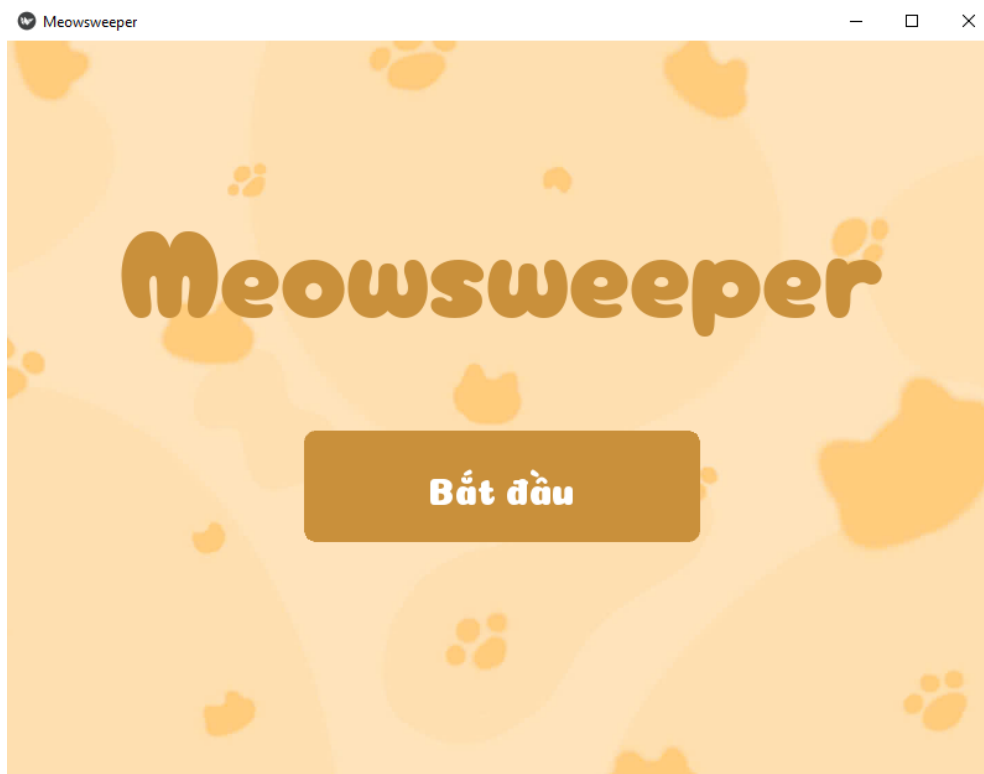
3. Kết quả đạt được

Đồ án đã phát triển thành công tựa game Meowsweeper với giao diện đồ họa tùy chỉnh, mang phong cách hiện đại, thân thiện và tự động co giãn tương thích với mọi kích thước màn hình. Về mặt hệ thống, trò chơi vận hành trơn tru với đầy đủ các cơ chế chuẩn mực của Dò mìn (Flood fill, Chording, First-click safe). Điểm nổi bật và mang tính kỹ thuật cao nhất của đồ án là việc tích hợp thành công hệ thống Trí tuệ nhân tạo (AI) hỗ trợ gợi ý nước đi. Bằng cách áp dụng mô hình Thỏa mãn Ràng buộc (CSP) kết hợp cùng thuật toán Vết cạn, AI có khả năng giải quyết chính xác các thế cờ khó mà không gây quá tải CPU. Đồ án đã mang lại một sản phẩm phần mềm hoàn thiện, mượt mà cả về mặt tối ưu thuật toán lẫn trải nghiệm người dùng.

4. Tài liệu tham khảo

[1] RaemonDBW, "Python-Minesweeper," GitHub repository. [Trực tuyến]. Địa chỉ: <https://github.com/RaemonDBW/Python-Minesweeper>

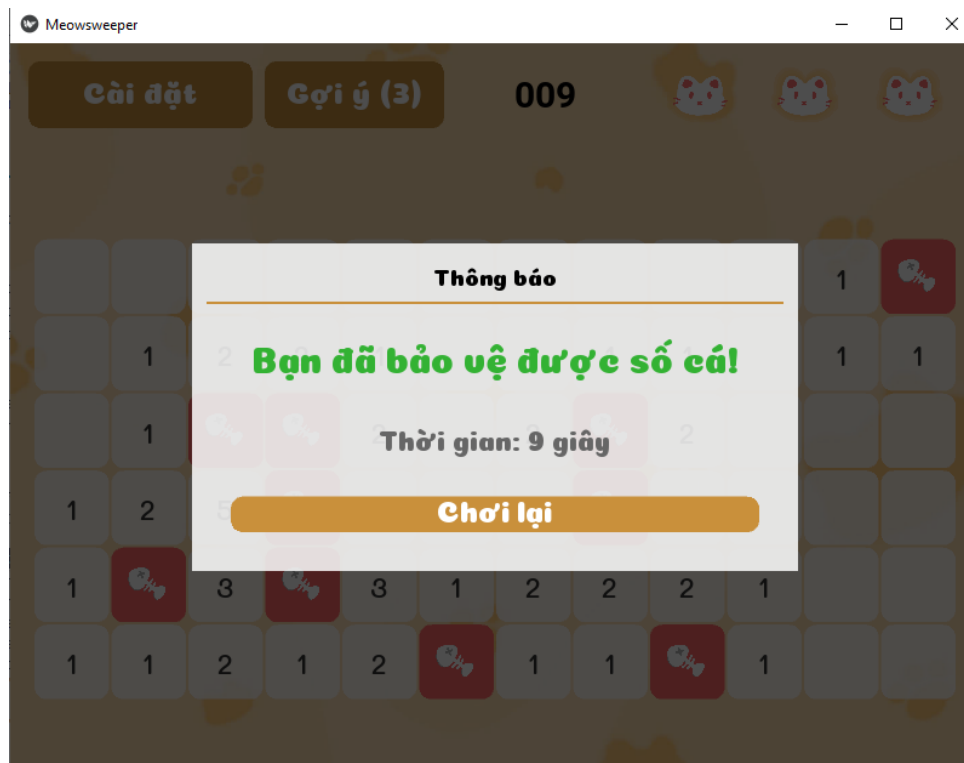
5. Phụ lục 1: Giới thiệu (demo) kết quả



Màn hình khởi động trò chơi



Màn hình gameplay



Màn hình khi thắng trò chơi

6. Phụ lục 2: docstring

=== CLASS: Cell ===

Lớp đại diện cho một ô vuông trên bảng Dò Mìn.

Attributes:

isMine (bool): Trạng thái ô có chứa mìn hay không (True = có mìn).

isRevealed (bool): Trạng thái ô đã được người chơi mở hay chưa.

isFlagged (bool): Trạng thái ô đã bị cấm cờ nghi ngờ có mìn.

neighbors (int): Số lượng mìn hiện diện trong 8 ô lân cận (từ 0 đến 8).

+ HÀM: `__init__()`

(Chưa có docstring)

=== CLASS: Board ===

Lớp đại diện cho bảng Dò mìn.

Attributes:

rows (int): Số hàng trong bảng chơi.

cols (int): Số cột trong bảng chơi.

num (int): Số lượng mìn được rải vào bảng chơi.

grid (list): Mảng 2 chiều chứa các đối tượng Cell đại diện cho bàn cờ.

lives (int): Số mạng sống hiện tại của người chơi.

game_over (bool): Trạng thái kết thúc trò chơi.

is_win (bool): Trạng thái người chơi đã chiến thắng hay chưa.

+ HÀM: `__init__()`

Khởi tạo bảng chơi với kích thước và số lượng mìn chỉ định.

Args:

rows (int): Số hàng của bảng.

cols (int): Số cột của bảng.

num (int): Tổng số mìn cần rải.

lives (int): Số mạng sống khởi đầu (mặc định là 3).

+ HÀM: `place_mines()`

Rải mìn ngẫu nhiên lên bảng chơi. Tránh ô được chọn đầu tiên và 8 ô lân cận để đảm bảo người chơi có đủ dữ kiện để giải màn chơi.

Args:

first_row (int): Tọa độ hàng của ô được chọn đầu tiên.

first_col (int): Tọa độ cột của ô được chọn đầu tiên.

Returns:

None. Gọi hàm `calculate_neighbors` sau khi rải xong.

+ HÀM: `calculate_neighbors()`

Duyệt qua toàn bộ bảng chơi để đếm và gán số lượng mìn xung quanh

(từ 0-8) cho từng ô không chứa mìn.

Returns:

None. Cập nhật thuộc tính 'neighbors' cho các đối tượng Cell.

+ HÀM: reveal()

Xử lý logic khi người chơi mở một ô. Sử dụng thuật toán đệ quy Flood Fill (DFS) để tự động mở lan truyền các ô an toàn lân cận nếu ô hiện tại có số mìn xung quanh là 0.
Quản lý trừ mạng nếu trúng mìn.

Args:

r (int): Tọa độ hàng của ô cần mở.

c (int): Tọa độ cột của ô cần mở.

Returns:

None. Cập nhật trạng thái 'isRevealed' của các ô và trừ 'lives' nếu trúng mìn.

+ HÀM: reveal_safe_around()

Hàm tự động lật mở tất cả các ô lân cận, nếu số cờ xung quanh hay số mìn đã mở bằng đúng neighbors của ô hiện tại.

Args:

r (int): Tọa độ hàng của ô được chọn.

c (int): Tọa độ cột của ô được chọn.

+ HÀM: check_win()

Kiểm tra điều kiện chiến thắng của trò chơi.

Người chơi thắng khi tổng số ô đã mở bằng đúng tổng số ô an toàn trên bảng.

Returns:

None. Cập nhật trạng thái 'is_win' và 'game_over' thành True nếu thắng.

+ HÀM: reveal_all()

Lật mở toàn bộ các ô chứa mìn trên bàn cờ.

Được gọi khi trò chơi kết thúc (dù thắng hay thua).

=== CLASS: Constraint ===

Lớp đại diện cho một mệnh đề/ràng buộc logic trong game.

Ví dụ: "Trong tập hợp 3 ô (A, B, C) này có chính xác 1 quả mìn".

Attributes:

cells (set): Tập hợp các tọa độ (r, c) của các ô chưa mở.

mines (int): Số lượng mìn chắc chắn nằm trong tập hợp các ô trên.

+ HÀM: __init__()

Khởi tạo một ràng buộc mới.

Args:

cells (list/set): Danh sách các ô liên quan đến ràng buộc.

mines (int): Số mìn nằm trong các ô này.

+ HÀM: is_empty()

Kiểm tra xem ràng buộc này đã rỗng (không còn ô nào) hay chưa.

Returns:

bool: True nếu tập hợp cells rỗng, ngược lại là False.

=== CLASS: MinesweeperAI ===

Lớp Trí tuệ nhân tạo (AI) giúp suy luận và giải game Dò Mìn.

Attributes:

rows (int): Số hàng của bàn cờ.

cols (int): Số cột của bàn cờ.

knowledge (list): Danh sách các đối tượng Constraint (Cơ sở tri thức).

safe (set): Tập hợp tọa độ các ô chắc chắn an toàn.

mines (set): Tập hợp tọa độ các ô chắc chắn là mìn.

processed (set): Tập hợp các ô đã được AI xử lý.

+ HÀM: `__init__()`

Khởi tạo bộ não AI với kích thước bàn cờ.

Args:

rows (int): Số hàng.

cols (int): Số cột.

+ HÀM: `add_knowledge()`

Cập nhật cơ sở tri thức của AI mỗi khi một ô mới được mở.

Nó sẽ quét các ô lân cận để tạo ra một Ràng buộc (Constraint) mới.

Args:

cell (tuple): Tọa độ (r, c) của ô vừa được mở.

number (int): Con số hiển thị trên ô đó (tổng số mìn xung quanh).

board (Board): Đối tượng bàn cờ hiện tại để lấy dữ liệu.

Returns:

None. Tri thức mới sẽ được lưu vào thuộc tính ``knowledge``.

+ HÀM: `infer()`

Thuật toán cốt lõi để suy luận logic từ Cơ sở tri thức hiện tại.

Thực hiện các bước:

1. Tìm các ràng buộc hiển nhiên (`mines == 0` hoặc `mines == số ô`).

2. Đơn giản hóa các ràng buộc cũ nếu đã biết chắc chắn ô nào là mìn/an toàn.
3. Áp dụng lý thuyết tập hợp (Subset Rule): Nếu tập A là con của tập B, ta có thể suy ra ràng buộc mới cho phần hiệu (B - A).

Returns:

None. Thuộc tính `safe`, `mines` và `knowledge` sẽ được cập nhật liên tục cho đến khi không thể suy luận thêm.

+ HÀM: `get_hint()`

Cung cấp gợi ý nước đi tiếp theo cho người chơi.

Args:

board (Board): Trạng thái bàn cờ hiện tại.

Returns:

tuple: Trả về một tuple có dạng (action_type, (r, c)).

- ("wrong_flag", (r, c)): Người chơi cắm cờ sai vị trí.
- ("safe", (r, c)): Có ô chắc chắn an toàn để mở.
- ("mine", (r, c)): Có ô chắc chắn là mìn để cắm cờ.
- ("none", None): Bế tắc, AI không thể suy luận chắc chắn.

+ HÀM: `brute_force()`

Thuật toán vét cạn có tối ưu (Backtracking). Được gọi khi hàm `infer()` bế tắc.

Hàm sẽ gom nhóm các ràng buộc giao nhau thành các Cụm (Components) độc lập.

Sau đó thử giả định mọi trường hợp mìn có thể xảy ra trong từng Cụm.

Nếu một ô LUÔN an toàn hoặc LUÔN là mìn trong mọi trường hợp hợp lệ, ô đó sẽ được đưa vào tri thức chuẩn xác.

Returns:

bool: True nếu tìm ra thêm ô an toàn hoặc mìn mới, ngược lại là False.

=== CLASS: CellUI ===

Lớp giao diện đại diện cho một nút bấm (ô) trên màn hình (View).

Kế thừa từ Button của thư viện Kivy. Quản lý hiển thị hình ảnh, màu sắc và bắt các sự kiện tương tác từ chuột/cảm ứng.

Attributes:

logic_cell (Cell): Tham chiếu đến đối tượng dữ liệu logic của ô này.

r (int): Tọa độ hàng của ô trên lưới.

c (int): Tọa độ cột của ô trên lưới.

app_ref (MeowsweeperApp): Tham chiếu đến ứng dụng gốc để gọi ngược lại các hàm xử lý.

bg_color (Color): Thuộc tính màu nền tùy chỉnh trên canvas.

bg_rect (RoundedRectangle): Khối hình chữ nhật bo góc làm nền.

icon_color (Color): Màu sắc/độ trong suốt của icon.

icon_rect (Rectangle): Khối hình chữ nhật chứa texture của icon (cờ/mìn).

+ HÀM: __init__()

Khởi tạo giao diện ô và thiết lập các thành phần đồ họa mặc định.

Args:

logic_cell (Cell): Đối tượng chứa dữ liệu logic.

r (int): Tọa độ hàng.

c (int): Tọa độ cột.

app_ref (MeowsweeperApp): Đối tượng app chính.

+ HÀM: update_rect()

Hàm callback tự động cập nhật lại kích thước và vị trí của các khối Canvas mỗi khi cửa sổ ứng dụng bị thay đổi kích thước.

+ HÀM: on_touch_down()

Xử lý sự kiện khi người chơi nhấn chuột xuống.

Chuyên dùng để bắt sự kiện click chuột phải để thực hiện thao tác cấm/rút cò.

Args:

touch (MotionEvent): Đối tượng chứa thông tin về thao tác chạm/click.

Returns:

bool: Trả về True nếu sự kiện đã được xử lý (ngăn Kivy lan truyền sự kiện), ngược lại gọi hàm gốc của lớp cha.

+ HÀM: on_release()

Xử lý sự kiện khi người chơi nhả chuột/ngón tay (tương đương click chuột trái).

- Nếu ô chưa mở: Gửi yêu cầu mở ô.

- Nếu ô đã mở và có số lân cận > 0: Gọi tính năng Chord (mở nhanh xung quanh).

Returns:

None.

+ HÀM: update_display()

Đồng bộ giao diện đồ họa của nút bấm để phản ánh đúng trạng thái logic.

Cập nhật màu nền, hiển thị số lân cận hoặc load hình ảnh (mìn/cá) tương ứng.

Returns:

None. Thay đổi trực tiếp các thuộc tính hiển thị của nút.

=== CLASS: RoundedSpinnerOption ===

Lớp giao diện đại diện cho từng lựa chọn thả xuống bên trong Menu Spinner.

Được tùy chỉnh Canvas để có các viền bo tròn mềm mại thay vì hình chữ nhật sắc cạnh.

Attributes:

bg_color (Color): Màu nền của lựa chọn.

bg_rect (RoundedRectangle): Hình khối bo góc hiển thị trên Canvas.

+ HÀM: `__init__()`
(Chưa có docstring)

+ HÀM: `update_rect()`
(Chưa có docstring)

=== CLASS: `RoundedSpinner` ===

Lớp giao diện tùy chỉnh cho Menu Thả xuống (Dropdown/Spinner) chính.

Được thiết kế để đồng nhất ngôn ngữ thiết kế bo góc với các nút bấm khác trong game.

Attributes:

`custom_color` (list/tuple): Màu sắc tùy chọn cho nút Spinner truyền vào dưới dạng RGBA.

+ HÀM: `__init__()`
Khởi tạo Spinner chính với màu sắc được cung cấp hoặc màu mặc định.

Args:

`custom_color` (list, optional): Mã màu RGBA. Mặc định là `#C9903BFF`.

`**kwargs`: Các tham số thuộc tính gốc của Kivy Spinner.

+ HÀM: `update_rect()`
Cập nhật động kích thước hình học dựa vào giao diện khung ứng dụng.

=== CLASS: `RoundedButton` ===

Lớp giao diện đại diện cho một nút bấm tương tác cơ bản nhưng được tùy chỉnh viền bo tròn.

Hỗ trợ thay đổi màu sắc linh hoạt thông qua tham số khởi tạo.

Attributes:

`custom_color` (list/tuple): Màu sắc nền của nút bấm dưới dạng mảng RGBA.

+ HÀM: `__init__()`

Khởi tạo nút bấm bo góc.

Args:

`custom_color` (list, optional): Mã màu nền RGBA. Mặc định là [1, 0.6, 0.8, 1].

`**kwargs`: Các tham số thuộc tính gốc của Kivy Button.

+ HÀM: `update_rect()`

Cập nhật động kích thước hình học dựa vào giao diện khung ứng dụng.

=== CLASS: `MeowsweeperApp` ===

Lớp ứng dụng cốt lõi (Controller). Quản lý vòng đời trò chơi,

khởi tạo giao diện (View), xử lý tương tác tổng thể và liên kết với Logic (Model).

Attributes:

`sm` (ScreenManager): Bộ quản lý chuyển đổi giữa các màn hình (Start, Game).

`game_board` (Board): Đối tượng chứa logic của bàn cờ.

`ai` (MinesweeperAI): Đối tượng Trí tuệ nhân tạo dùng để gợi ý nước đi.

`ui_grid` (list): Mảng 2 chiều chứa các đối tượng CellUI (nút bấm giao diện).

`time_elapsed` (int): Thời gian đã trôi qua tính bằng giây.

`timer_event` (ClockEvent): Đối tượng quản lý sự kiện lặp bộ đếm thời gian.

`hints_left` (int): Số lượt sử dụng gợi ý còn lại.

+ HÀM: `build()`

Khởi tạo hệ thống giao diện, thiết lập các màn hình và kết nối các phần tử UI với Logic.

Returns:

ScreenManager: Trả về bộ quản lý màn hình gốc chứa toàn bộ giao diện ứng dụng.

+ HÀM: `go_to_game()`

Hàm callback chuyển từ màn hình Start sang màn hình Game.

Args:

instance (Button): Nút bấm phát sinh sự kiện.

+ HÀM: create_grid_ui()

Tạo mới hoặc vẽ lại toàn bộ lưới giao diện bàn cờ.

Được gọi khi bắt đầu ứng dụng hoặc khi chọn chơi lại/đổi độ khó.

+ HÀM: update_lives_ui()

Cập nhật hiển thị số lượng trái tim (mạng sống) trên thanh công cụ.

+ HÀM: on_menu_select()

Xử lý sự kiện khi người dùng chọn một mục trong menu thả xuống (Cài đặt).

Args:

spinner (Spinner): Đối tượng menu thả xuống.

text (str): Giá trị mà người dùng vừa chọn.

+ HÀM: reset_game()

Khởi tạo lại toàn bộ trạng thái logic và giao diện của trò chơi.

Args:

rows (int): Số hàng mới.

cols (int): Số cột mới.

mines (int): Tổng số mìn mới.

+ HÀM: give_hint()

Thu thập dữ liệu từ bàn cờ, cung cấp cho bộ não AI để suy luận

và hiển thị nước đi gợi ý tiếp theo lên giao diện.

Args:

instance (Button): Nút bấm Gọi ý.

+ HÀM: `handle_chord()`

Thực hiện tính năng mở nhanh (Chording) khi người dùng click vào ô đã có số.
Nếu số cờ lân cận bằng với số trên ô, tự động mở tất cả các ô xung quanh.

Args:

r (int): Tọa độ hàng.

c (int): Tọa độ cột.

+ HÀM: `handle_click()`

Hàm trung gian nhận yêu cầu mở ô từ CellUI và ra lệnh cho Board xử lý.
Đảm bảo rải mìn an toàn ở click đầu tiên và kích hoạt kiểm tra Thắng/Thua.

Args:

r (int): Tọa độ hàng được click.

c (int): Tọa độ cột được click.

+ HÀM: `update_timer()`

Hàm callback được gọi mỗi giây bởi Kivy Clock để cập nhật bộ đếm thời gian.

Args:

dt (float): Delta time (Khoảng thời gian trôi qua giữa 2 lần gọi hàm).

+ HÀM: `sync_ui_with_logic()`

Đồng bộ toàn bộ giao diện (View) để phản ánh trạng thái mới nhất của Bộ não (Model).
Cập nhật số mạng sống hiện tại, làm mới màu sắc/ảnh của tất cả nút bấm,
và gọi hiển thị thông báo popup nếu game đã kết thúc.

+ HÀM: `update_board_size()`

Tự động tính toán lại kích thước hiển thị của lưới (Grid)

khi cửa sổ thiết bị thay đổi kích thước, đảm bảo các ô luôn là hình vuông hoàn hảo.

+ HÀM: `show_endgame_popup()`

Tạo và hiển thị cửa sổ nổi (Popup) khi trò chơi kết thúc.

Cung cấp thông báo động dựa trên kết quả Thắng/Thua và thời gian hoàn thành.

Args:

`is_win` (bool): True nếu người chơi chiến thắng, False nếu thua cạn mạng.

+ HÀM: `restart_from_popup()`

Xử lý sự kiện khi người chơi bấm nút "Chơi lại" trên Popup kết thúc game.

Tắt Popup và gọi hàm khởi tạo màn chơi mới.

Args:

`instance` (Button): Nút bấm phát sinh sự kiện.